Automatic Index Creation in PostgreSQL

Phase 2 Project Report

Mahathi Nakka 23B0965 | Gehna Chelvi Burri 23B1012 | Aditi Tapase 23B1047 | Akkshitha Rathod 23B1070

1. Motivation

In PostgreSQL, indexes are created manually. If nobody creates an index on a frequently queried column, the database reads every single row in the table for every query on that column. This is called a sequential scan and it gets very slow for large tables.

We built a system that watches how queries run and automatically creates an index when it becomes worth it. If the same column is getting scanned again and again, at some point the total time wasted on those scans is more than the one-time cost of building an index. That is when we create one.

Professor Suraj introduced the **ski rental problem** in Phase 1 as the right way to think about this. The idea is: if you rent skis every day, after enough days you have spent more than the cost of just buying them. In our case, each sequential scan is like paying rent, and building an index is like buying the skis. We keep a running total of how much the scans have cost, and the moment that total exceeds the one-time cost of building an index, we build it.

2. Dataset

We used the **Olist Brazilian E-commerce** dataset from Kaggle for all our testing. It contains real order, customer, product, and review data from a Brazilian online marketplace. We loaded five tables into PostgreSQL:

- `olist_customers` – around 99,000 rows, with columns like `customer_state` and `customer_city`
- `olist_orders` – around 99,000 rows, with a column like `order_status`
- `olist_order_items` – around 112,000 rows, with a column like `price`
- `olist_products` – around 32,000 rows
- `olist_reviews` – around 99,000 rows, with `review_score` which only has 5 distinct values

We chose this dataset because the columns have realistic distributions. For example, `customer_state` has 27 distinct values, which is a good candidate for indexing. `review_score` has only 5 distinct values, which is a good test for making sure we do not create a useless index.

3. Files Created and Modified

We modified the PostgreSQL 16.0 source. We created two new files and changed four existing ones.

File	What we did
`nodeSeqscan.c` (modified)	This is where the bulk of our work lives. We added new includes, global shared memory pointers, and the following functions entirely written by us: `auto_index_shmem_init()` to set up shared memory at startup; `read_guc_double()` and `read_guc_long()` to read cost settings from `pg_settings`; `compute_thresholds()` and `ensure_thresholds_fresh()` to keep decision thresholds up to date; `get_pred_col_attnum()` to detect equality and range predicates in the WHERE clause; `find_or_create_slot()` to manage the shared memory array; `index_already_exists()` to check for duplicate indexes; `estimate_selectivity()` to read column statistics from `pg_statistic`; `record_scan_stat()` which is the main hook containing the ski rental decision logic; and `record_write_stat()` to count write operations. We also added one call to `record_scan_stat(node)` inside the existing `ExecSeqScan()` function.
`execModifyTable.c` (modified)	Added `record_write_stat()` calls in `ExecInsert`, `ExecUpdate`, and `ExecDelete` to count writes per table.
`postmaster.c` (modified)	Added a block to register our background worker so it starts when the database starts.
`postmaster/Makefile` (modified)	Added `auto_index_bgworker.o` to the build.
`auto_index.h` (new)	Shared header with our two main structs, constants, and function declarations.
`auto_index_bgworker.c` (new)	Background worker that wakes every 10 seconds and runs `CREATE INDEX` or `DROP INDEX`.

The main data structure is `ScanStat`. We keep one per (table, column) pair in PostgreSQL shared memory, so all database connections update the same counters. It stores accumulated scan cost, scan count, write count, last scan time, and the OID of any index we created. A single `LWLock` protects it from race conditions.

A second struct, `AutoIndexThresholds`, holds values read from `pg_settings` – `seq_page_cost`, `random_page_cost`, and `checkpoint_timeout`. In Phase 1 we had hardcoded numbers for thresholds. Now all thresholds are computed from these real settings so the system works correctly on different hardware. For example, on an SSD `random_page_cost` is much lower, so the system creates indexes more readily. These values are refreshed every 60 seconds.

4. Features Implemented

4.1 Detecting Predicates – Equality and Range (Features 1 and 4)

Every time a sequential scan runs, `record_scan_stat()` looks at the WHERE clause. PostgreSQL stores it as a list of operator expressions internally. We go through each one and check: is one side a column and the other a constant value? If yes, and the operator is =, >, <, >=, or <=, we record the column and start tracking it. Range predicates are included because B-tree indexes support them. `BETWEEN a AND b` also works because PostgreSQL rewrites it into two separate conditions.

4.2 Ski Rental Decision (Feature 2)

Every scan adds to the running cost: $\Delta_1 \mathrel{+}= N \times 0.01$, where N is the row count from `pg_class.reltuples`. We compare this to the one-time cost of building a B-tree:

$$\text{ICT} = N \times \log_2(N) \times R$$

where R is `random_page_cost`. When $\Delta_1 \geq \text{ICT}$, we flag the slot and the background worker creates the index. For the demo we divided ICT by 100 so the threshold could be crossed in about 7 queries.

4.3 Write Penalty (Feature 3)

We count writes per table and before creating any index check: $\text{writes}/(\text{writes} + \text{scans}) \leq S/(S + R \cdot \log_2(N)/N)$. If the write fraction is too high, we skip creation. On our 100k-row tables the threshold is near 100%, so it did not visibly fire during testing.

4.4 Selectivity Guard (Feature 5)

We read `stadistinct` from `pg_statistic` and estimate selectivity $= 1/\texttt{stadistinct}$. If this exceeds S/R, we skip the column. On HDD the cutoff is 0.25, on SSD it rises to 0.91.

4.5 Redundancy Check (Feature 6)

We wrote code in the background worker to check `pg_index` before creating an index to avoid duplicates. However, we could not get this to work correctly during testing. The check was never triggered in a way we could observe or verify. We were not able to identify the exact cause.

4.6 Background Index Creation (Feature 7)

`record_scan_stat()` only sets a flag. The actual index creation happens in our background worker. Every 10 seconds it checks for flagged slots and runs `CREATE INDEX IF NOT EXISTS`. We wanted `CREATE INDEX CONCURRENTLY` to avoid locking the table, but PostgreSQL does not allow it inside a transaction block and SPI always runs inside one.

4.7 Recency Decay (Feature 8)

We implemented code to halve accumulated scan cost for each full `checkpoint_timeout` period with no scans: $\Delta_1 \mathrel{\times}= 0.5$. However when we tested this by reducing `checkpoint_timeout` and waiting, we did not see the expected decay messages in the logs. We were not able to figure out what was going wrong.

4.8 Dropping Stale Indexes (Feature 9)

We wrote a second pass in the background worker to check `pg_stat_user_indexes.idx_scan` and drop any auto-created index unused for over an hour. However, during testing the drop never happened and no related log messages appeared. We are not sure whether the issue is in how we read the stats view or how we compute the time difference.

5. Testing

Build and setup commands used (and what they do):

- `./configure --prefix=/usr/local/pgsql` – configures the PostgreSQL build to install into our chosen directory
- `make -j4` – compiles the source using 4 parallel jobs to speed it up
- `sudo make install` – installs the compiled binaries into `/usr/local/pgsql`
- `initdb -D /usr/local/pgsql/data` – creates a fresh database cluster with empty data files
- `pg_ctl start` – starts the PostgreSQL server as a background process
- `echo "log_min_messages = debug1" >> postgresql.conf` – enables debug-level logs so we can see our `elog(DEBUG,...)` messages
- `tail -f logfile | grep auto_index` – watches the live log and filters only our system's messages

What we tested:

1. Ran `EXPLAIN ANALYZE` first to confirm the plan says `Seq Scan` with no index.
2. Ran 7 repeated equality queries on `olist_customers.customer_state`, waited 10 seconds, and confirmed `auto_idx_olist_customers_customer_state` appeared in `pg_indexes`. Running `EXPLAIN ANALYZE` again showed `Index Scan` and query time dropped from 18 ms to 0.3 ms.
3. Repeated with range predicates on `olist_order_items.price > 100.00` and saw the same result, with the log showing `predicate: range`.
4. Ran scans on `olist_reviews.review_score`, which has 5 distinct values (selectivity = 0.20, threshold = 0.25 on HDD). No index was created and the log showed the selectivity guard firing.
5. Reduced `checkpoint_timeout` to 30 seconds and waited, but did not see the decay messages we expected in log.
6. Set `AUTO_INDEX_STALE_SECONDS = 120` and waited beyond the timeout, but the background worker did not drop the index.

Server log when index creation was triggered:

```
LOG: auto_index: table OID 16469 col 5 (equality predicate): scan cost 9840.00 >=
     index build cost 9600.00 (N=99441, log2N=16.6, R=4.00). Signaling index creation.
LOG: auto_index: creating index: CREATE INDEX IF NOT EXISTS
     auto_idx_olist_customers_customer_state ON olist_customers (customer_state)
LOG: auto_index: successfully created auto_idx_olist_customers_customer_state
```

6. Use of AI

We used ChatGPT and Claude throughout the project. Below is an honest account of what we used them for and what we did ourselves.

6.1 Prompts We Used

Ski rental formula:

```
"Our Professor told us to use the ski rental problem to decide when to create
an index in PostgreSQL. Each sequential scan is like paying rent and building the
index once is buying. How do we write this as a concrete formula? What should the
index build cost be for a B-tree on N rows, expressed using PostgreSQL cost
parameters like random_page_cost and seq_page_cost?"
```

Shared memory design:

```
"We want to store an array of structs in PostgreSQL shared memory so that all
backend processes (one per client connection) can read and write the same counters.
How do we allocate this at startup using ShmemInitStruct()? Should we use LWLock
or a spinlock to protect it? What is the difference and which one is better for
our use case where reads happen very frequently and writes are occasional?"
```

Reading pg_statistic for selectivity:

```
"How do I read the stadistinct value for a specific (table OID, column attnum)
pair from pg_statistic in PostgreSQL C code? I do not want to use SPI and run
a SELECT -- I want to use the system cache directly. Which SearchSysCache call
should I use and how do I extract the float4 value from the returned HeapTuple?"
```

Background worker setup:

```
"How do I register a PostgreSQL 16 background worker that starts automatically
when the database starts, has access to shared memory, and can connect to a
specific database to run SQL via SPI? Show me the exact BackgroundWorker struct
fields I need to set in postmaster.c and how the worker function should initialize
itself before entering its main loop."
```

Write penalty formula:

```
"We want to skip creating an index if a table has too many writes relative to
reads, because every INSERT/UPDATE/DELETE also updates the index. How do we
derive a break-even write ratio from seq_page_cost and random_page_cost? We
want a formula that accounts for the B-tree leaf update cost per write."
```

Debugging the integer cast bug:

```
"Our code uses IsA(right, Const) to detect the right-hand side of a predicate
like col > constant. It correctly detects WHERE price > 100.00 but silently
misses WHERE price > 100, even though both look the same to us. The column type
is NUMERIC. Why does PostgreSQL treat an integer literal differently from a
decimal literal in this context at the plan node level?"
```

CONCURRENTLY transaction error:

```
"Our PostgreSQL background worker uses SPI_execute() to run CREATE INDEX
CONCURRENTLY and we get the error: ERROR: CREATE INDEX CONCURRENTLY cannot run
inside a transaction block. But our worker does StartTransactionCommand() before
any SPI call, which seems required. Is there any way to run CONCURRENTLY from
inside a background worker, or is this a fundamental incompatibility with SPI?"
```

Predicate detection in the executor:

```
"In PostgreSQL 16's executor, when ExecSeqScan() runs, how do I get the list of
WHERE clause conditions (quals) for that scan? I have a SeqScanState* node. I
want to walk the qual list, find OpExpr nodes, and check whether the left or right
argument is a Var (column reference) and the other is a Const (literal value).
What is the correct way to check the operator OID to identify equality vs range?"
```

6.2 What We Did Not Use AI For

We did not use AI for everything. Several parts we researched and figured out ourselves:

The ski rental math: After ChatGPT gave us the formula, we verified the $\log_2(N)$ derivation ourselves using the Wikipedia article on B-tree complexity and the PostgreSQL documentation on cost estimation. We wanted to make sure we understood it before using it, not just copy it.

Understanding PostgreSQL cost parameters: We read the official PostgreSQL 16 (`https://www.postgresql.org/docs/16/runtime-config-query.html`) to understand what `seq_page_cost` and `random_page_cost` actually mean and how the planner uses them. AI gave us function names but the conceptual understanding came from reading the docs ourselves.

Loading the dataset and writing the test SQL: We wrote `test_auto_index.sql` ourselves. We read the Olist dataset documentation on Kaggle to understand the schema and chose the right columns to test each feature on.

Build system and compilation: We figured out the Makefile changes, configure flags, and common build errors (missing readline, missing libz) ourselves by reading error messages and the PostgreSQL installation guide.

Resources we used without AI:

- PostgreSQL 16 source code comments in `nodeSeqscan.c`, `execModifyTable.c`, and `postmaster.c`
- PostgreSQL documentation: *Chapter 20: Server Configuration*, *Chapter 55: Writing a Background Worker Process*
- `src/backend/postmaster/autovacuum.c` – we read this to understand how autovacuum registers as a background worker and modeled our registration code on it
- Olist dataset schema documentation on Kaggle

7. Known Limitations

Integer cast bug. Queries like `WHERE price > 100` with an integer literal against a NUMERIC column are not detected because PostgreSQL wraps the integer in a cast node. Writing `WHERE price > 100.00` works. We know the fix but did not implement it.

No CONCURRENTLY. SPI runs inside a transaction so `CREATE INDEX CONCURRENTLY` is not possible. Index creation briefly locks the table.

Write penalty does not fire on large tables. The formula gives a threshold near 100% for 100k-row tables, so it only fires for very small write-heavy tables.

Redundancy check, recency decay, and stale index dropping did not work. We wrote the code for all three but could not get any of them to produce the expected output during testing. We could not identify the exact bugs before submission.

8. Conclusion

All project files are available at: https://github.com/Gehnachelvi-7/DBIS-PROJECT

We built a working automatic index creation system inside PostgreSQL 16.0. The system watches sequential scans, uses the ski rental formula from Professor Suraj's suggestion to decide when an index is worth building, avoids useless indexes using the selectivity guard, and builds indexes in the background. All thresholds come from real hardware settings rather than hardcoded numbers. The main result: a query on `olist_customers.customer_state` dropped from 18 ms to 0.3 ms after the system automatically created an index. The selectivity guard worked as expected. Features 6, 8, and 9 were implemented in code but did not produce verifiable results during testing. AI tools helped us understand PostgreSQL internals and debug specific C-level issues. The design decisions, the ski rental formula, and the verification of the math all came from the course material, Professor Suraj's guidance in Phase 1, and our own reading of the PostgreSQL documentation.